

Framework de Galpões de Aço

Catálogo dos 55 módulos - o que cada um realmente é

Visão geral

O framework realiza o projeto estrutural paramétrico e ponta-a-ponta de galpões de aço: dados do terreno → cargas (vento/sismo/ponte) → análise do pórtico 2D → 2ª ordem → dimensionamento NBR 8800 → ligações → fundações → fogo/acessórios → modelo 3D no FreeCAD → pranchas 2D executivas (TechDraw) + memoriais PDF/DXF. Tudo com princípio de **zero-erro-de-método**: os valores de norma são lidos diretamente do PDF, nunca de memória.

1. Orquestração e infraestrutura

Módulo	O que realmente é
<code>framework.py</code>	Ponto de entrada. Descobre a raiz do repo por <code>__file__</code> (sem caminho absoluto), carimba a versão nos memoriais e cria pastas de projeto isoladas (zero contaminação entre projetos).
<code>projeto_spec.py</code>	Contrato de dados do projeto - fonte única da verdade. <code>validar()</code> bloqueia cálculo e desenho enquanto houver campo não decidido. Mappers <code>to_rodar_params</code> / <code>to_build_kwargs</code> traduzem o spec para os módulos.
<code>wizard.py</code>	[NOVO] Entrada guiada via console (perguntas interativas ou presets) que constrói e valida o <code>ProjetoSpec</code> .
<code>rodar_galpao.py</code>	Orquestrador da cadeia de cálculo (Gates 5-9). Configura os módulos, roda na ordem dos gates e extrai os esforços do próprio pórtico (não hardcoded) - serve para qualquer geometria.
<code>rodar_projeto.py</code>	Runner limpo de um projeto: reset -> validar -> cálculo -> modelo. Portável e isolado; também dispara as pranchas executivas.
<code>build_final.py</code>	Script utilitário de build/execução completa de um projeto exemplo.
<code>smoke_executivo.py</code>	Teste de fumaça que constrói 8 geometrias de galpão ponta-a-ponta (headless) e verifica que cada uma gera as pranchas sem erro.

2. Análise estrutural (motor)

Módulo	O que realmente é
<code>frame2d.py</code>	Solver de pórtico plano 2D pelo método da rigidez direta. Entra geometria/seções/cargas -> sai deslocamentos e esforços (N, V, M) por barra.
<code>galpao_portico.py</code>	Monta e analisa o pórtico do galpão (1 ou N vãos geminados) via <code>frame2d</code> ; aplica os casos de carga.
<code>estabilidade_b1b2.py</code>	2ª ordem aproximada (MAES, NBR 8800 Anexo D). Decompõe cada combinação em nt/lt, calcula B1/B2 e amplifica esforços por grupo de seções.
<code>redimensionamento.py</code>	Auto-sizing guloso: parte do perfil mais leve, sobe um degrau na coluna mais solicitada e itera. Cada coluna pode ter perfil diferente (vãos assimétricos).

3. Verificação de perfis (NBR 8800 / aço)

Módulo	O que realmente é
<code>check_nbr8800.py</code>	Verificação núcleo NBR 8800:2008 de perfis I/H (Nsd, Msd, Vsd): compressão com flambagem global (χ) + local ($Q=Q_s*Q_a$), flexão (Anexos F/G).
<code>perfis.py</code>	Biblioteca de propriedades de perfis I laminados (IPE/HEA/HEB) in SI.
<code>props_I_mono.py</code>	Propriedades de perfil I monossimétrico (mesas diferentes) - habilita o ramo mono do AISC DG25.
<code>alma_esbelta.py</code>	Momento resistente de vigas de alma esbelta (NBR 8800 Anexo H, quando $\lambda = h/t_w > 5,70 \sqrt{E/f_y}$).
<code>forcas_localizadas.py</code>	Mesas/almas sob forças transversais localizadas + enrijecedor de apoio (NBR 8800 Seção 5.7).
<code>tensao_ponto.py</code>	Verificação por tensões da elasticidade (NBR 8800 5.5.2.3) - para o joelho tapered onde M e V picam juntos.

4. Perfis de alma variável / treliça (pórticos avançados)

Módulo	O que realmente é
<code>alma_variavel.py</code>	Perfil I de alma variável (tapered): gera seções ao longo do comprimento para alimentar o frame2d in NSEG segmentos.
<code>tesoura.py</code>	Treliça de cobertura (tesoura) - banzo reto em duas águas, isostática, tipo Warren; método dos nós.
<code>flt_misula.py</code>	FLT de barras de seção variável (mísula) pelo método normativo NBR 8800 Anexo J (não o AISC).
<code>dg25_ltb.py</code>	Cross-check informativo da FLT de mísula pelo AISC Design Guide 25 (não altera o dimensionamento, só confere).
<code>cortante_tapered.py</code>	Cortante da alma em barra tapered (mesas inclinadas carregam parte do cortante -> alma vê cortante efetivo menor).
<code>enrijecedor_painel.py</code>	Enrijecedores transversais da alma do painel do joelho (NBR 8800 5.4.3.1; kv sobe de 5).
<code>zona_painel.py</code>	Zona de painel do nó rígido viga-coluna (joelho): painel de alma do pilar + estados locais -> decide doubler/enrijecedor.

5. Cargas (ações)

Módulo	O que realmente é
<code>vento_nbr6123.py</code>	Vento NBR 6123/1988: S2, V _k , q, coeficientes C _{pe} /C _{pi} (paredes Tab.4, telhado Tab.5), transversal + longitudinal.
<code>sismo_nbr15421.py</code>	Sismo NBR 15421:2023 pelo método das forças horizontais equivalentes (constantes lidas do PDF).
<code>ponte_rolante.py</code>	Ação de ponte rolante (NBR 8800 + NBR 8400): cargas, verificação da viga de rolamento e reação empacotada para o pórtico.
<code>nbr8400.py</code>	Classificação e coeficientes da NBR 8400-1:2019 (fornece os números que a ponte_rolante pedia a confirmar).
<code>neve.py</code>	Carga de neve em coberturas (EN 1991-1-3) integrado ao pipeline: caso simétrico atua como Q _{efetivo} da coberta e caso assimétrico é sinalizado.

6. Peças secundárias e telha

Módulo	O que realmente é
<code>tercas_iteracao.py</code>	Itera perfis Ue e adota a terça mais leve que passa (NBR 14762), com propriedades pela linha média (pycufsm).
<code>tercas_nbr14762.py</code>	Verificação de terça formada a frio (Ue) pela NBR 14762:2010 (genérico).
<code>distorcional_fsm.py</code>	Momento de flambagem distorcional (Mdist) de perfil Ue pelo método das faixas finitas (FSM) que a NBR 14762 exige e não dá fechado.
<code>secundarios_nbr8800.py</code>	Longarina de parede (girt), escora e montante de oitão - flexão biaxial NBR 8800.
<code>mao_francesa.py</code>	Contenção lateral da mesa inferior (flange brace) - cria ponto de travamento contra FLT (Anexo G).
<code>contraventamento.py</code>	Barras tracionadas (contraventamento só-tração + tirantes/sag rods) - leva o arrasto do vento à fundação.
<code>telha_cobertura.py</code>	Telha vencendo o vão entre terças (ELU flexão NBR 14762 + sucção de vento).
<code>calhas.py</code>	Calhas e condutores (Bellei / NBR 10844): vazão, seção (Manning), diâmetro.
<code>junta_dilatacao.py</code>	Necessidade de junta de dilatação e movimento térmico longitudinal.

7. Ligações

Módulo	O que realmente é
<code>ligacoes.py</code>	Núcleo de ligações (parafusadas/soldadas) NBR 8800 - primitivos: parafusos, solda de filete, block shear, esmagamento.
<code>gusset_ligacao.py</code>	Chapa de gusset dos contraventamentos - compõe os primitivos do <code>ligacoes.py</code> .
<code>console_ponte.py</code>	Ligação do console da ponte rolante à coluna (chapa/solda que recebe a viga excêntrica).
<code>base_chumbador.py</code>	Ligação de base (placa + chumbadores) sob N/V/M - base rotulada ou engastada.

8. Fundações

Módulo	O que realmente é
<code>fundacao_sapata.py</code>	Sapata isolada (geotecnia/estabilidade + concreto armado NBR 6118).
<code>viga_baldrame.py</code>	Viga de baldrame/amarração entre sapatas (NBR 6118): sob alvenaria + tração de amarração.
<code>estaca_profunda.py</code>	Fundação profunda: capacidade da estaca por Aoki-Velloso (SPT) + bloco de coroamento.
<code>sapata_divisa.py</code>	Sapata de divisa rasa com viga alavanca (Velloso & Lopes / NBR 6122).
<code>viga_equilibrio.py</code>	Bloco sobre estacas na divisa + viga de equilíbrio (variante profunda da fundação de divisa).

9. Fogo e acessórios

Módulo	O que realmente é
fogo_nbr14323.py	Dimensionamento ao fogo (NBR 14323:2013): temperatura do aço após TRRF, resistências reduzidas, proteção intumescente/spray.
escada.py	Escadas industriais (longarinas, degraus, patamares, guarda-corpo) - NBR 8800/6120, NR-18, Blondel.
plataforma.py	Plataformas e passarelas (vigas, piso, guarda-corpo, flecha L/350 e frequência > 3 Hz).

10. Geometria, terreno e saídas

Módulo	O que realmente é
terreno.py	Passo zero: viabilidade do terreno (lê lote em KML/coordenadas, área, parâmetros urbanísticos que limitam o galpão).
build_galpao.py	Modelo 3D paramétrico no FreeCAD - todos os elementos em português; gap de graute, placas, escoras, montantes, terças, contraventamento; auditoria de interferências.
techdraw_exec.py	Projeto executivo 2D via TechDraw headless: pranchas A1 (ISO 5457) com cortes hachurados e símbolos de solda AWS A2.4 -> PDF/SVG/DXF/PNG.
relatorio_calculo.py	Memorial PDF consolidado - cada módulo precedido do método (norma + procedimento) para conferência do engenheiro sênior.
dossie.py	[NOVO] Junta num único PDF entregável (caderno de projeto): capa + relatório consolidado + memorial de cálculo + pranchas executivas.

11. Bugs e Gaps Técnicos Identificados (Auditoria Continuada)

Esta seção apresenta as inconsistências, gaps e bugs encontrados na auditoria do framework. Atualizado após a sessão de correções do desenvolvedor e as rodadas de fuzzing avançado.

Área / Módulo	Erro / Gap Identificado e Status de Correção
frame2d.py <i>(Sinal do UDL)</i>	[CORRIGIDO] A carga nodal equivalente da UDL estava com sinal invertido (subtraída no vetor de forças). Isso fazia as cargas gravitacionais agirem como arrancamento (uplift). Resolvido pelo desenvolvedor no frame2d.py e testado no test_frame2d_sinal.py.
projeto_spec.py <i>(Carga de Parede)</i>	[CORRIGIDO] O peso do fechamento de parede era coletado no spec e ignorado no cálculo. Agora está integrado no orquestrador como UDL vertical nas colunas externas e repassado para a sapata e para a viga de baldrame (parâmetro q_parede). Testado no test_carga_parede.py.
projeto_spec.py <i>(Janela Lateral)</i>	[CORRIGIDO] O wizard entregava largura/altura (L,H), enquanto o build 3D esperava uma faixa vertical (z_base, z_topo). Corrigido via helper aberturas_para_build(). Testado no test_aberturas_janela.py.
projeto_spec.py <i>(Mapeador de Terreno)</i>	[CORRIGIDO] Os parâmetros urbanísticos inseridos no wizard eram ignorados pelo to_rodar_params(), silenciando o Gate 10. Corrigido pelo desenvolvedor e adicionado o modo de checagem simplificada por área (recuos pendentes). Testado no test_terreno_mapper.py.
galpao_portico.py <i>(Vento 1 Vão)</i>	[REMANESCENTE / PENDENTE - CRÍTICO] O vento de telhado em _wind_unico() é aplicado como força vertical para baixo (wy = -wy_udl) e o Cpi de pressão interna positiva também atua para baixo, ignorando totalmente o \$C_{pe}\$ da Tabela 5 da NBR 6123. Isso anula o arrancamento (uplift) por sucção, gerando subdimensionamento crítico na tração de fundações/ancoragens.
projeto_spec.py <i>(Roteamento Alvenaria)</i>	[REMANESCENTE / PENDENTE - GAP] Todo o peso da parede de fechamento (incluindo alvenaria pesada de blocos autoportantes) é transferido como UDL vertical para as colunas de aço. Embora conservador para as colunas, não reflete o fluxo real onde a alvenaria descarrega diretamente no baldrame e sapatas, sem sobrecarregar o pórtico de aço.
rodar_galpao.py <i>(KeyError 'Hvr')</i>	[REMANESCENTE / PENDENTE - NOVO BUG] O orquestrador tenta acessar pcfg["Hvr"] por indexação direta na linha 216. No entanto, Hvr não é um campo obrigatório no bloco ponte em projeto_spec.py. Se o spec não tiver explicitamente a chave, o programa quebra com KeyError: 'Hvr' e aborta a execução.
projeto_spec.py <i>(Validação de Solo)</i>	[REMANESCENTE / PENDENTE - NOVO BUG] O validador de dados apenas checa se o perfil_spt está preenchido, mas não checa se os tipos de solo são válidos. Se o usuário fornecer um tipo não suportado pelo motor Aoki-Velloso (ex: 'rocha'), a execução quebra com ValueError: tipo de solo invalido durante o cálculo.
rodar_galpao.py <i>(IndexError de Reprovação)</i>	[REMANESCENTE / PENDENTE] Quando a estrutura é reprovada no auto-sizing, a lista res["perfil_colunas"] é inicializada com tamanho `nv` em vez de `nv+1`. Isso provoca um crash de IndexError: list index out of range na verificação final de colunas no loop de range(nv+1), interrompendo o pipeline.
build_galpao.py <i>(Variáveis Globais)</i>	[REMANESCENTE / PENDENTE] As variáveis globais SPAN, TOTAL_Y, RIDGE_Y e RIDGE_H são estáticas e nunca são recalculadas quando a função configurar() é invocada. Elas permanecem fixadas nos valores de importação.
build_galpao.py <i>(Terças/Fechamentos)</i>	[REMANESCENTE / PENDENTE] Girts e fechamentos da 'Parede D' dependem do global SPAN. Em vãos personalizados (ex: Preset 3 de 12 m), estas peças são geradas a 10m, ficando flutuando a 2 metros de distância da parede real.
build_galpao.py <i>(Ligação Cumeeira)</i>	[REMANESCENTE / PENDENTE] A cumeeira_conn() desenha a chapa de cumeeira nas globais estáticas RIDGE_Y/H. No multi-vão, todas as cumeeiras são criadas de forma sobreposta na primeira posição, deixando os demais vãos sem chapas e parafusos de cumeeira.

Multi-Vão Heterogêneo (Conflito Geométrico)	[REMANESCENTE / PENDENTE] Divergência entre modelo matemático e 3D. No 2D (galpao_portico.py), todas as cumeeiras são geradas no mesmo nível RIDGE , achatando a inclinação de vãos maiores. No 3D (build_galpao.py), a inclinação é mantida a 10%, gerando um desalinhamento de 0.25 m no segundo ápice entre os modelos.
requirements.txt (Dependências)	[REMANESCENTE / PENDENTE] As bibliotecas reportlab (memorial PDF) e pymupdf / fitz (gerador do dossiê) continuam ausentes no arquivo [requirements.txt], impedindo instalações limpas.
dossie.py (Legenda de Capa)	[REMANESCENTE / PENDENTE] A geração da capa assume vãos idênticos baseando-se em spans[0]. Para múltiplos vãos de larguras diferentes, o relatório exhibe incorretamente '2 vaos de 10 m'.

Como se conecta: projeto_spec guarda os dados → rodar_projeto / rodar_galpao chamam os módulos de cálculo na ordem dos gates → build_galpao gera o 3D → techdraw_exec gera as pranchas → relatorio_calculo fecha o memorial → dossie.py consolida a entrega turn-key num único PDF. Os módulos tapered/mísula, fundações de divisa, ponte e sismo só disparam se o projeto tiver esses elementos.